



**RAJALAKSHMI ENGINEERING COLLEGE,
CHENNAI**

**Department of Artificial Intelligence & Data Science
CS23633 – Cloud Computing Laboratory**

Experiment 2: Setting up Public Cloud

AIM:

To develop and deploy a simple web application using **Render**, a **Platform as a Service (PaaS)** cloud platform, and understand how cloud platforms host applications without requiring users to manage the underlying servers and infrastructure.

PROCEDURE

1. Create a project folder named **RenderHelloApp** on the local computer and create two files, **app.py** and **requirements.txt**.

```

app - Notepad
File Edit Format View Help
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello! Welcome to Render PaaS."

@app.route("/about")
def about():
    return "This application is deployed on Render."

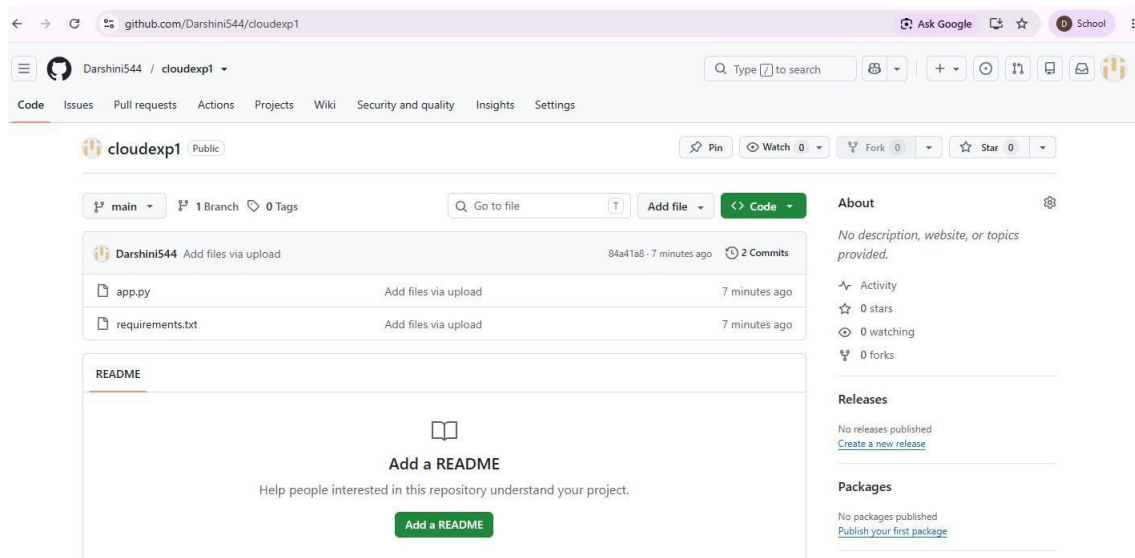
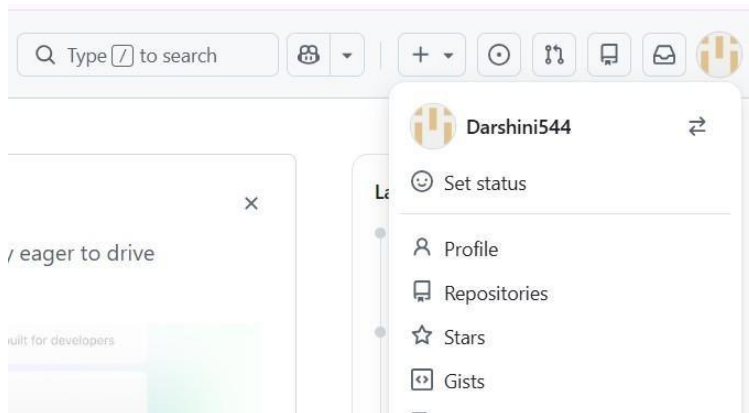
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
  
```

```

requirements - Notepad
File Edit Format View Help
Flask==3.0.3
gunicorn==22.0.0
  
```

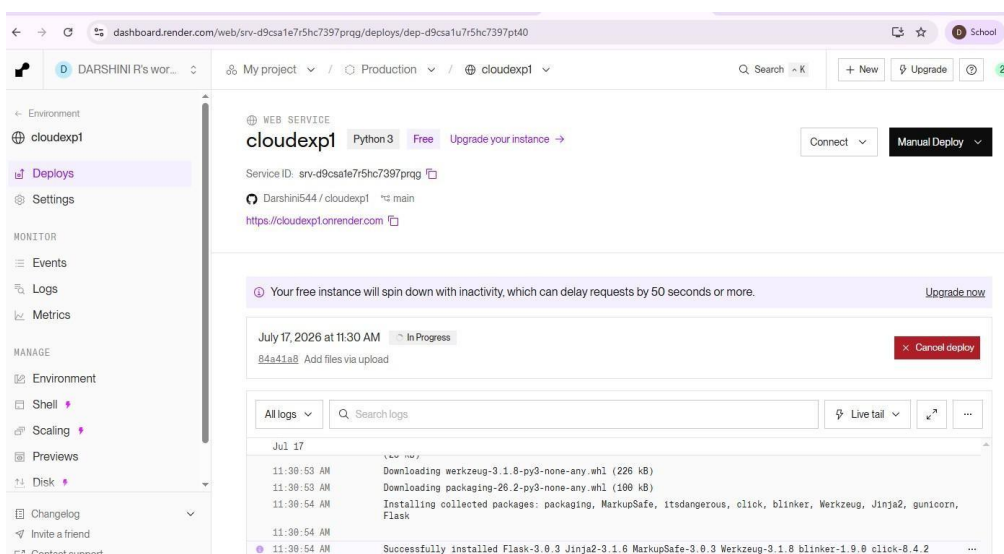
— □

2. Write the Flask application code in **app.py** to display a welcome message on the home page and an information message on the /about page.
3. Add the required Python dependency, **Flask** and **Gunicorn**, in the **requirements.txt** file.
4. Sign in to **GitHub** and create a new repository named **cloudexp1** with public visibility.

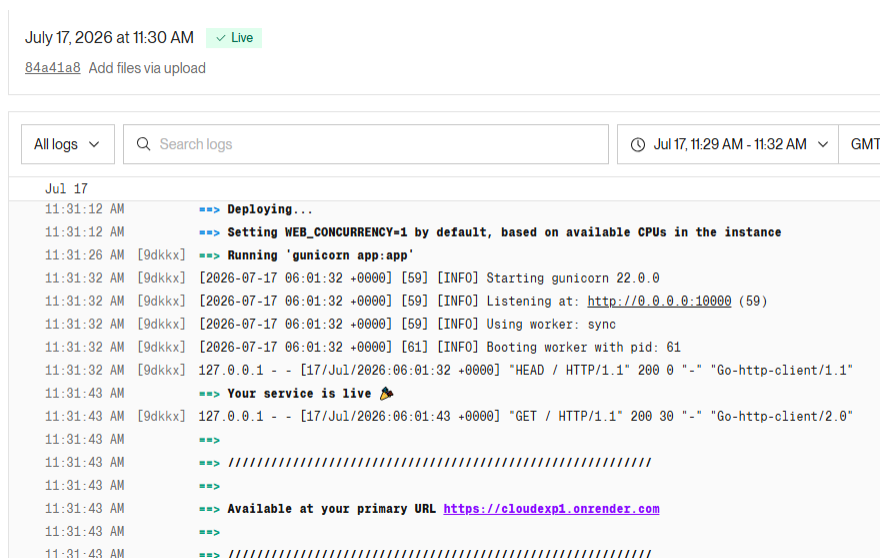


5. Upload **app.py** and **requirements.txt** to the GitHub repository and click **Commit changes** to save the files.

6. Open **Render** and sign in using the GitHub account.



- From the Render dashboard, select **Web Service** to deploy the Flask web application.
- Connect the GitHub account to Render and select the **cloudexp1** repository containing the application files.
- Configure the web service with the **main** branch, leave the Root Directory blank, and select **Python** as the environment.
- Set the **Build Command** as `pip install -r requirements.txt` and the **Start Command** as `gunicorn app:app`, then select the available free instance type.
- Click **Deploy Web Service** and wait for Render to build and deploy the application. Verify that the

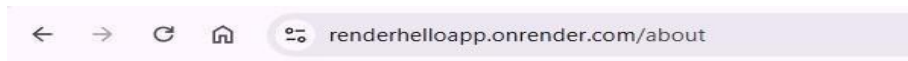


deployment status shows **Live**.

12. Open the generated Render URL in a web browser and verify that the home page displays “**Hello! Welcome to Render PaaS.**” Then open the /about URL and verify the corresponding message.
13. Test both pages and capture screenshots of the **Render Dashboard**, **Home Page**, and **About Page** as evidence of successful deployment.



Hello! Welcome to Render PaaS.



This application is deployed on Render.

CONCLUSION

Thus, a simple Flask web application was successfully developed and deployed using **Render PaaS**. The experiment demonstrated how a PaaS platform automatically manages the server environment, software installation, deployment, and hosting, allowing applications to be accessed through a public web URL without directly managing the underlying infrastructure.

